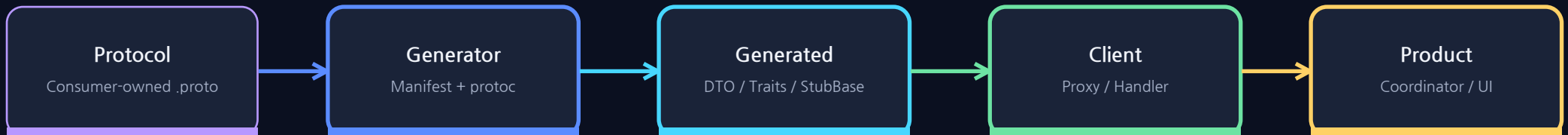


프로토콜 제작부터 Unreal Client 구현까지

ProjectSF의 실제 구조를 기준으로, .proto 정의가 어떻게 생성 코드가 되고 Runtime/Handler/Coordinator/UI까지 연결되는지 설명하는 포트폴리오 가이드입니다.



Portfolio Guide

Unreal Engine 5.8

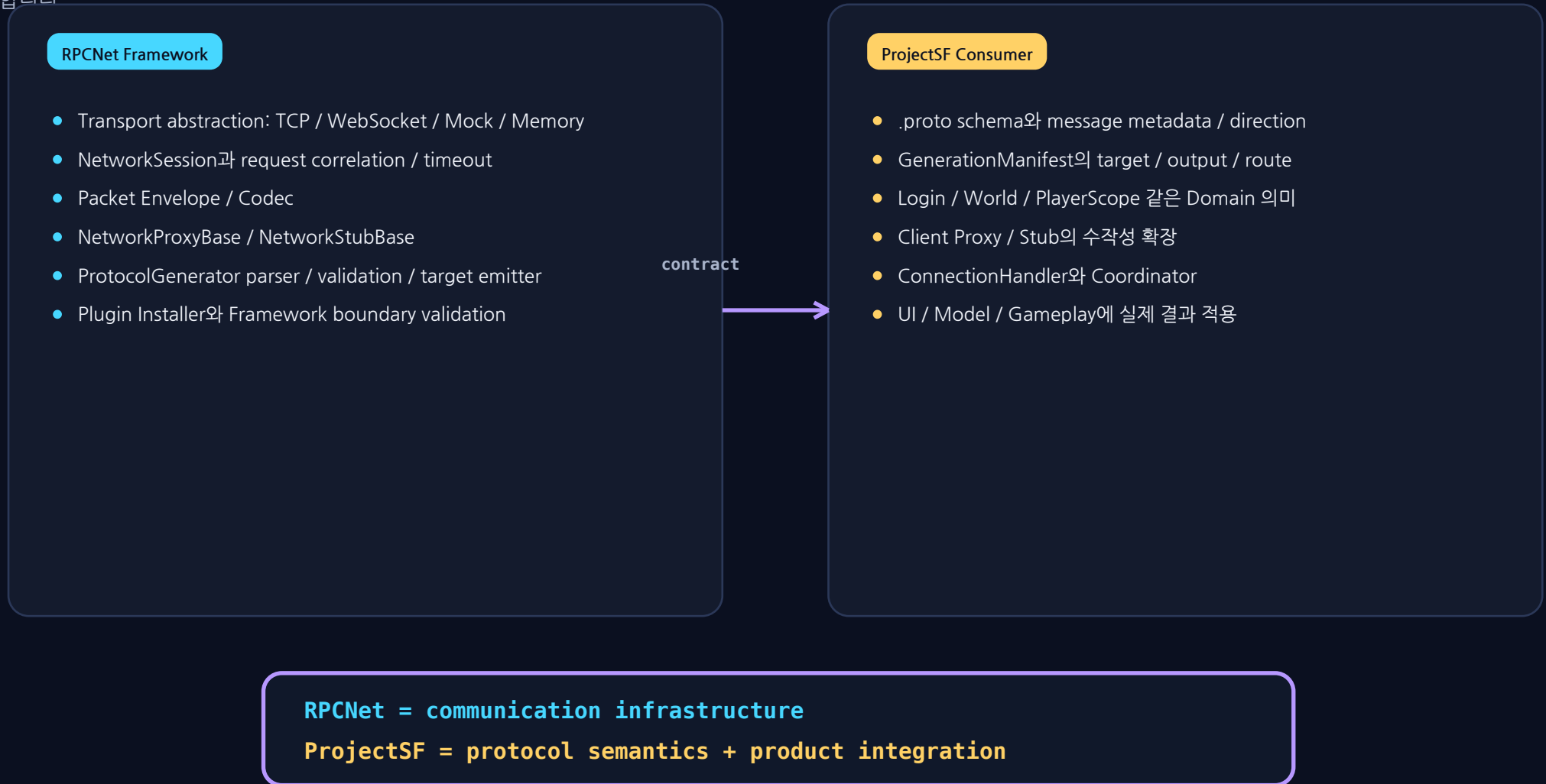
RPCNet

Real Client Flow

핵심: 생성 코드와 수작성 코드를 분리하고, Protocol 변경을 실제 게임 상태로 안전하게 연결한다.

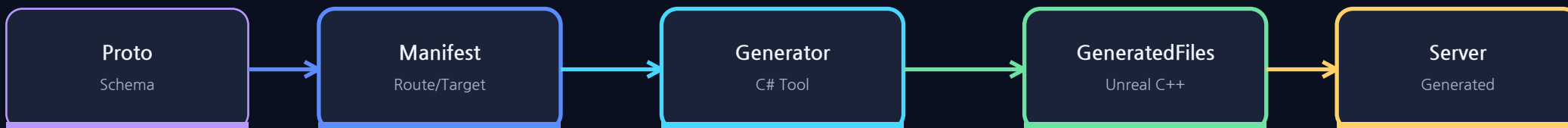
1. 먼저 경계를 나눈다

RPCNet이 프로토콜의 “의미”까지 소유하지 않는 것이 출발점입니다. Framework는 통신과 생성 인프라를 제공하고, ProjectSF가 실제 게임 Protocol과 Client 동작을 소유합니다



2. .proto -> Unreal Client 생성물

실제 ProjectSF generation manifest



Consumer-owned configuration

```

GenerationManifest.json
target: UnrealClient
output: Client/Source/SF/Network/GeneratedFiles
includePrefix: Network/GeneratedFiles

routes:
  C2AS -> AS2C
  C2GS -> GS2C
  GS2AS -> AS2GS
  Control2Server -> Server2Control
  
```

Generation entrypoint

```

Generate-Protocols.ps1 -Target Client

dotnet run --project ProtocolGenerator.csproj -- generate
--config GenerationManifest.json
--protoc <UE protobuf protoc.exe>
--protobuf-include <UE protobuf include>

Output examples:
C2ASProtocol.pb.h / .cc
AS2CAccountStubBase.h / .cpp
ProtocolTraits.generated.h
  
```

Generated source는 “소유권이 명확한 산출물”이다.

State/ClientGeneratedFiles.json이 생성기가 소유하는 파일을 추적하고, stale cleanup도 그 범위 안에서만 수행합니다. 생성물을 직접 수정하지 않습니다.

3. 생성 코드와 수작성 코드의 경계

GENERATED - 수정 금지

GeneratedFiles/

- protobuf message classes (.pb.h / .pb.cc)
- ProtocolTraits.generated.h
- AS2CAccountStubBase / GS2CGameStubBase
- Generator가 소유하는 metadata / dispatch glue

예시 역할

```
class FAS2CAccountStubBase : ...
{
    // generated dispatch
    virtual void OnRecvAccountStatus(...) = 0;
};

// Regenerate instead of editing this file.
```

extends

HANDWRITTEN - 제품 코드

Client/Source/SF/Network/

- Proxy: 제품 친화적 request API
- Stub: notify 수신을 delegate / model event로 변환
- ConnectionHandler: Session / Proxy / Stub lifecycle
- Coordinator: 여러 연결과 제품 Flow를 orchestration

예시 역할

```
class FAS2CAccountStub final
    : public FAS2CAccountStubBase
{
protected:
    void OnRecvAccountStatus(...) override;
};

// Stable handwritten extension point.
```

4. Unreal Client에서는 계층별로 책임을 올린다

Protocol이 생성됐다고 UI가 바로 protobuf를 호출하지 않습니다. Wire-level API를 제품 흐름으로 단계적으로 끌어올립니다.



5. Request / Notify를 제품 API로 감싼다

Request: Proxy

Handwritten product-facing request wrapper

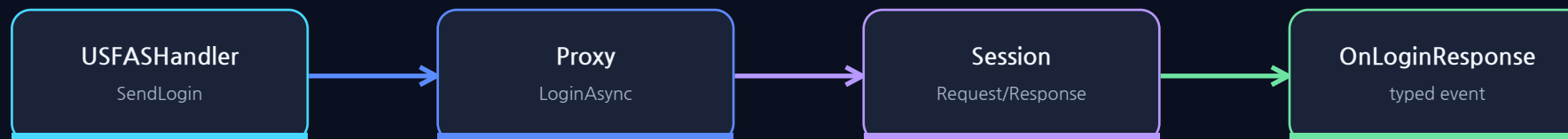
```
FPacketRequestHandle FC2ASAccountProxy::LoginAsync(
    const FString& AccountId, const FString& Password,
    FLoginCompletion&& Completion, double Timeout) const
{
    LoginRequest Request;
    Request.set_account_id(...);
    Request.set_password(...);
    return SendRequest<LoginRequest, LoginResponse>(
        Request, Timeout, MoveTemp(Completion));
}
```

Notifv: Stub

Generated dispatch -> handwritten event

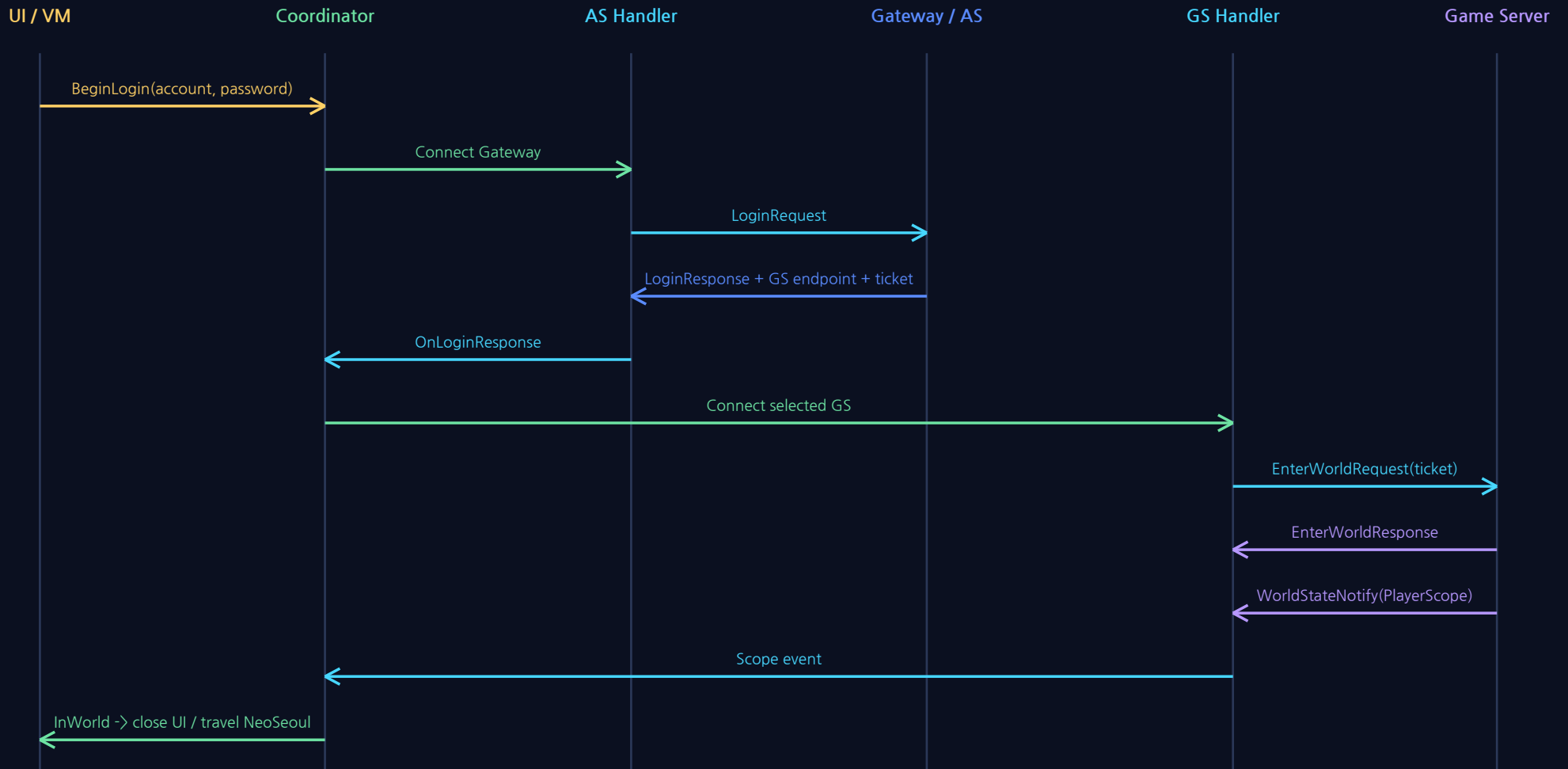
```
class FGS2CGameStub final
    : public FGS2CGameStubBase
{
public:
    FSFWorldStateReceived OnWorldState;
protected:
    void OnRecvWorldState(
        const WorldStateNotify& Data) override;
};
```

Handler는 연결별 Runtime을 소유한다



핵심은 protobuf 타입을 숨기는 것이 아니라 “어디까지가 wire contract이고 어디부터가 product behavior인지” 책임을 고정하는 것입니다.

6. 실제 로그인은 두 연결을 Coordinator가 이어 붙인다



AS는 handoff 후 종료 가능 / GS는 World 진입 이후 유지 / initial PlayerScope가 적용된 뒤에만 Travel

7. 네트워크는 “성공 경로”보다 수명주기가 중요하다

Correlation

AttemptId / ConnectionId / RequestId를 구분하고 과거 callback은 무시한다.

Timeout

Connect 5s / Login·EnterWorld RPC 10s / Initial Scope 12s. 무한 대기하지 않는다.

Cleanup

dispatch 중 session을 즉시 파괴하지 않고 event 처리 뒤 정리한다. Disconnect는 수렴 가능해야 한다.

Security / Handoff

Password·ticket·session id를 Evidence에 출력하지 않는다. GS ticket은 일회용으로 취급한다.

실패/단절 -> AuthModel + Scope + owned connection reset -> 새 Login 필요

nonzero ResultCode -> ProtocolError / 중복 EnterWorld -> reject

8. 새 Client Protocol을 추가하는 실제 순서

- 01 **Schema** Proto에 message / id / direction / response metadata 정의
- 02 **Generate** Generate-Protocols.ps1 -Target Client 실행
- 03 **Review generated** pb + Traits + StubBase 변경을 확인. 직접 수정 금지
- 04 **Proxy / Stub** request wrapper와 notify adapter를 handwritten layer에 추가
- 05 **Handler** 연결별 Session / Proxy / Stub lifecycle과 typed event 연결
- 06 **Product flow** Coordinator / Domain Service / ViewModel에서 실제 제품 동작 연결
- 07 **Build & tests** 반복 생성 hash, C++ build, failure / lifecycle test
- 08 **Real interop** 실제 Server process + UE Client readback으로 Acceptance 확인

Regression rule Protocol generation PASS != Client behavior PASS. 최소한 build, handler lifecycle, 실제 request/notify, product state 적용을 분리해서 검증합니다.

9. ProjectSF에서 실제로 확인한 것

Stable regeneration

동일 protocol 재생성 hash 확인

SFEditor build

UHT / C++ / DLL PASS

Real widget flow

입력/버튼 -> Gateway -> GS -> Model readback

Entry travel

Entry -> PlayerScope -> NeoSeoul

Failure / lifecycle cases

- 잘못된 비밀번호, login failure, GS disconnect
- cancel 이후 늦게 도착한 callback 무시
- 중복 world entry 거절, relogin과 server restart
- PlayerScope identity / session / revision correlation
- UI teardown과 Handler / Coordinator 수명 분리

현재 명시적으로 미검증

- Public TLS / WAN Unreal Client
- Packaged Game end-to-end
- 사람이 직접 입력하는 최종 화면 UX
- GS의 Unreal gameplay replication
- Production multi-region / HA

완료 판정 원칙

Generated file 존재나 RPC 호출 성공을 제품 완료로 보지 않고, 실제 Client state와 사용자 경험에 필요한 Acceptance까지 분리해 Evidence로 남깁니다.

10. 이 구조가 포트폴리오에서 보여주는 것

Protocol 변경을 “파일 생성”으로 끝내지 않는다.

Schema에서 출발한 변경이 Generated boundary, Runtime session, Handler lifecycle, Product Coordinator, UI/Model 적용, 실제 Server interoperability까지 이어지는 전체 경로를 설계하고 검증합니다.

Generate

반복 boilerplate를 자동화하되 generated ownership을 명확히 한다.

Integrate

wire-level contract를 Handler / Coordinator를 통해 제품 흐름으로 올린다.

Verify

실제 Server + Unreal Client까지 end-to-end Evidence를 남긴다.

핵심 설계 문장

Generated code is replaceable. Product behavior is handwritten and testable.

Protocol -> Runtime -> Handler -> Coordinator -> Product State